

Step 1: Requirements & Constraints

Functional Requirements

- Users create and 'like' posts.
- Search by keyword.
- Sort results by Recency or Like Count.
- Out of Scope: Fuzzy matching, personalization, images.

Non-Functional Requirements

- Low Latency: Median queries < 500ms.
- Freshness: New posts indexed < 1 minute.
- High Availability: Eventual Consistency acceptable.

THE CONSTRAINT: Prohibited: Elasticsearch, Lucene, or pre-built search engines. Goal: Build the index mechanism from scratch.

Scale Estimations: The Math

1 Billion

JetBrains Mono

Daily Active Users

Inter Regular

10k / sec
JetBrains Mono



Writes
(Posts)

Inter Regular

100k / sec



Writes
(Likes)

Inter Regular



Key Insight: The system is write-heavy due to engagement. Likes dominate the traffic (10x Posts).
Inter Regular



Storage

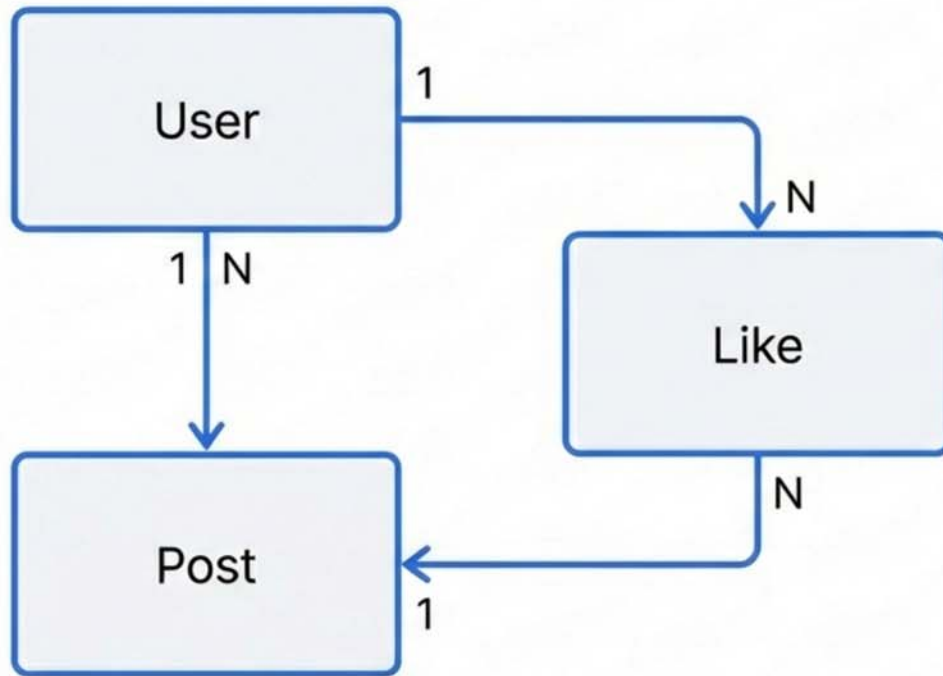
Inter Regular

3.6 PB (10 years)

JetBrains Mono

Step 2: Core Entities & API Design

Core Entities (ERD)



API Design

POST /posts

Request: { content: string }

POST /posts/{id}/likes

Request: { action: 'like' | 'unlike' }

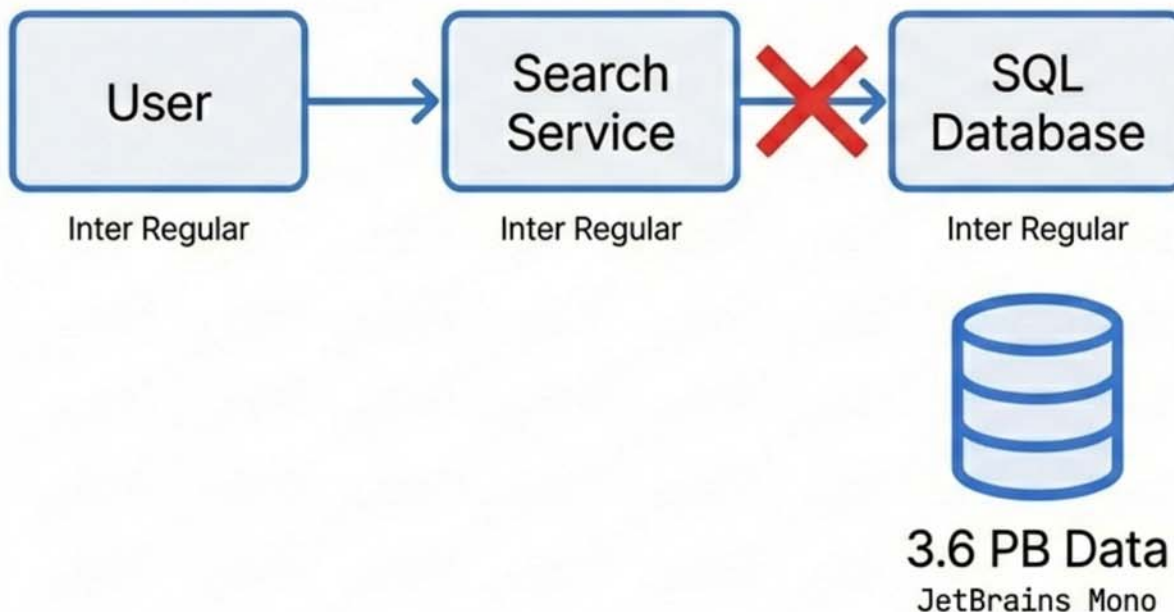
GET /posts

Query Params:

?key={keyword}

&sort={time | likes}

Step 3: The 'Naive' Approach (Why SQL Fails)



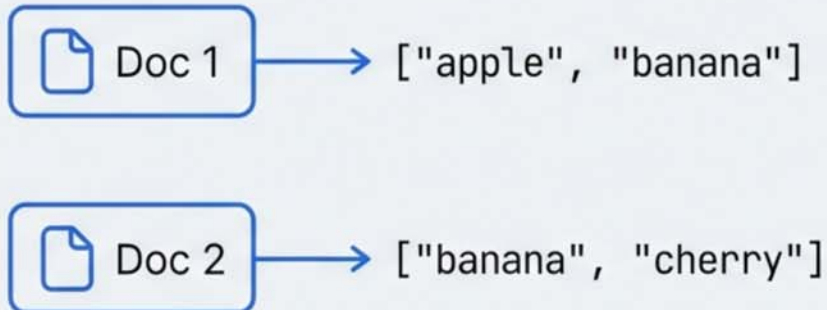
```
SELECT * FROM posts  
WHERE content LIKE '%keyword%'  
JetBrains Mono
```

Full Table Scan = Timeout.

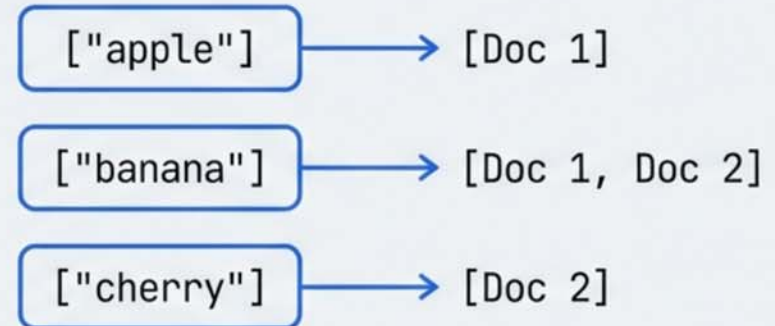
Scanning 3.6 PB of data for every search request is mathematically impossible under 500ms. We need an **index** optimized for reads.

Step 3: The Inverted Index Solution

Forward Index



Inverted Index (The Solution)

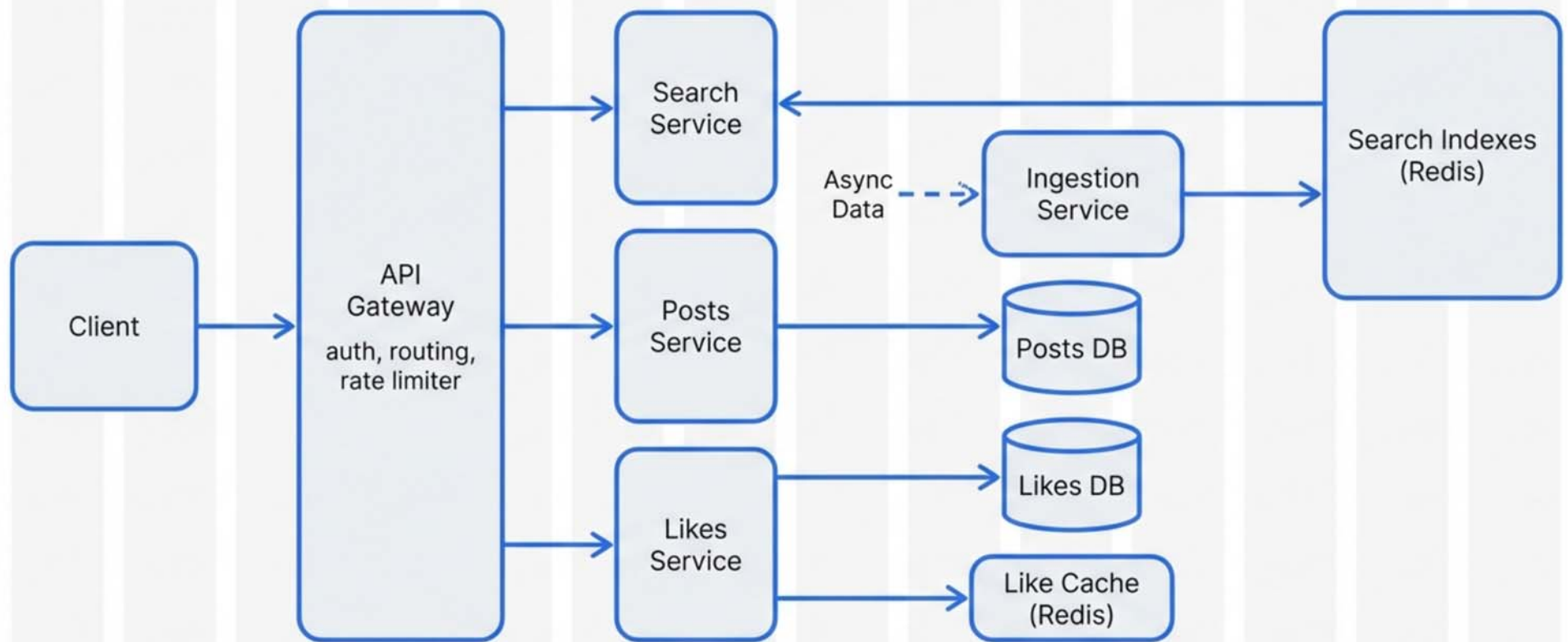


Implementation: Redis Sorted Sets

To support sorting requirements:

- Key: ``search:{keyword}:time`` -> Score: Timestamp
- Key: ``search:{keyword}:likes`` -> Score: LikeCount

Step 3: The Architecture Diagram



Step 4: Deep Dives (Identifying Bottlenecks)



Scaling Reads

The “Taylor Swift” Problem. 1M users search the same keyword simultaneously.



Scaling Writes

The “Like” Storm. 100k likes/sec triggers constant re-indexing.



Storage Costs

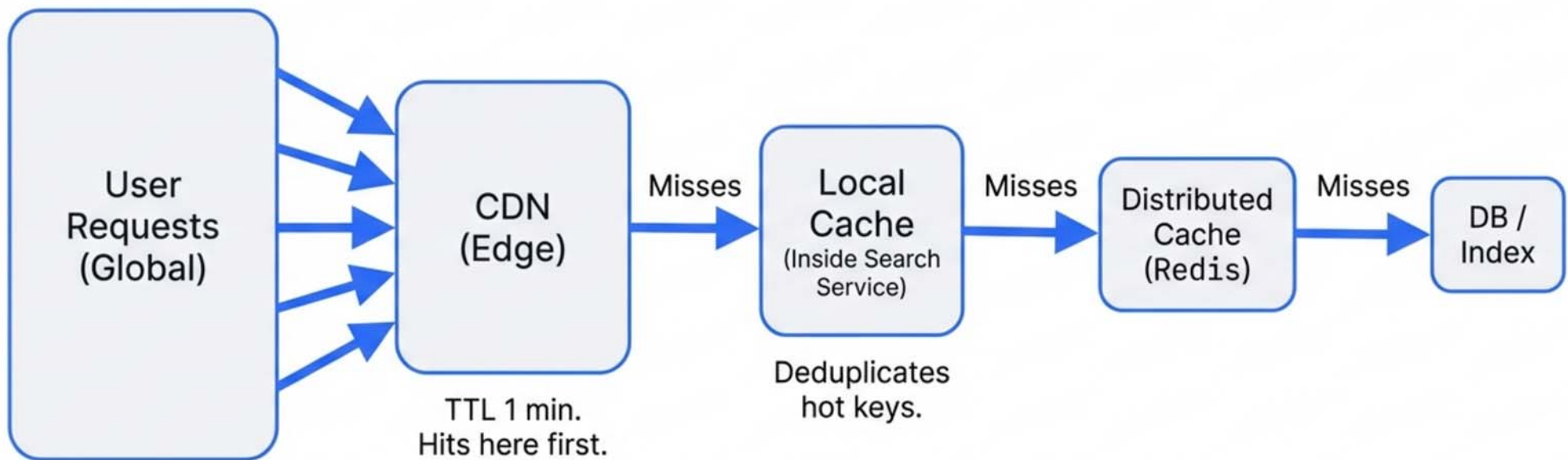
3.6 PB of data. Storing 10 years of history in RAM (Redis) is cost-prohibitive.



Multi-Keyword Search

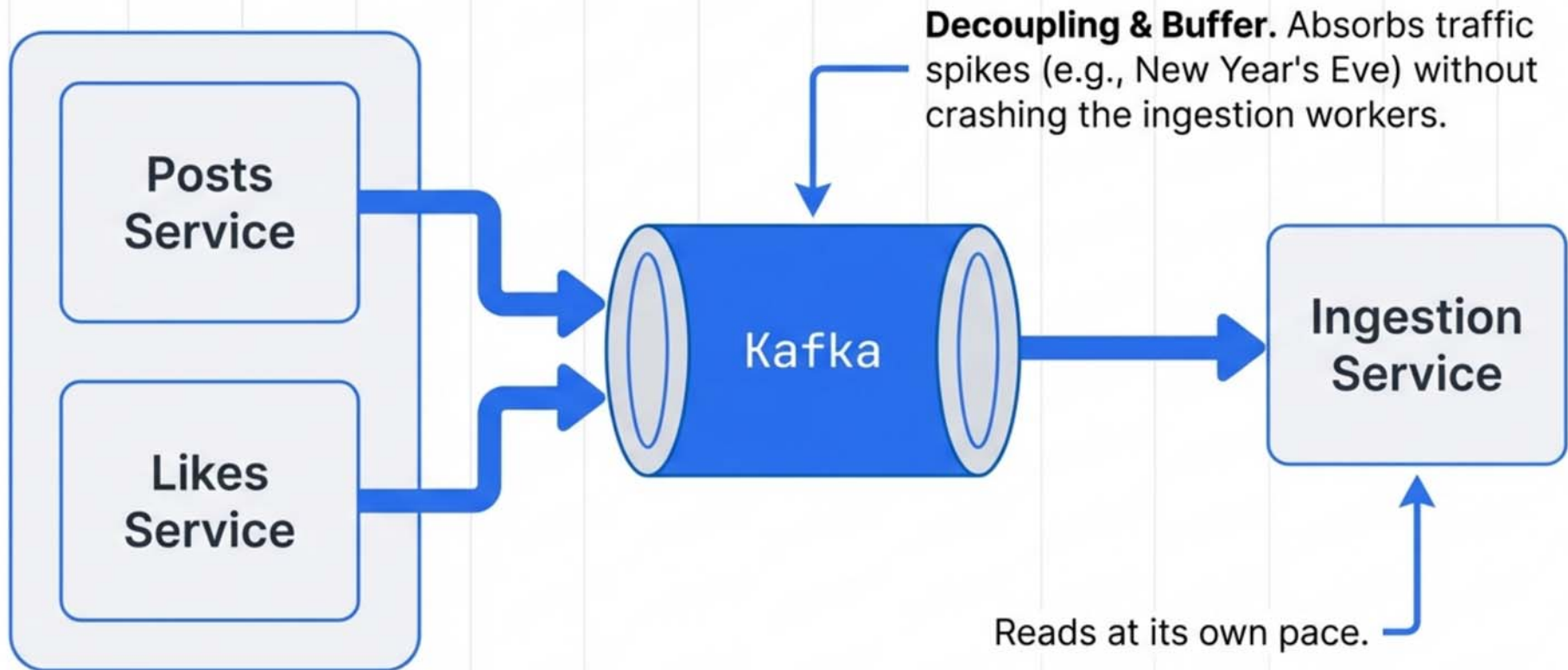
Handling “Taylor” AND “Swift” intersection efficiently.

Deep Dive 1: Scaling Reads (Caching Layers)

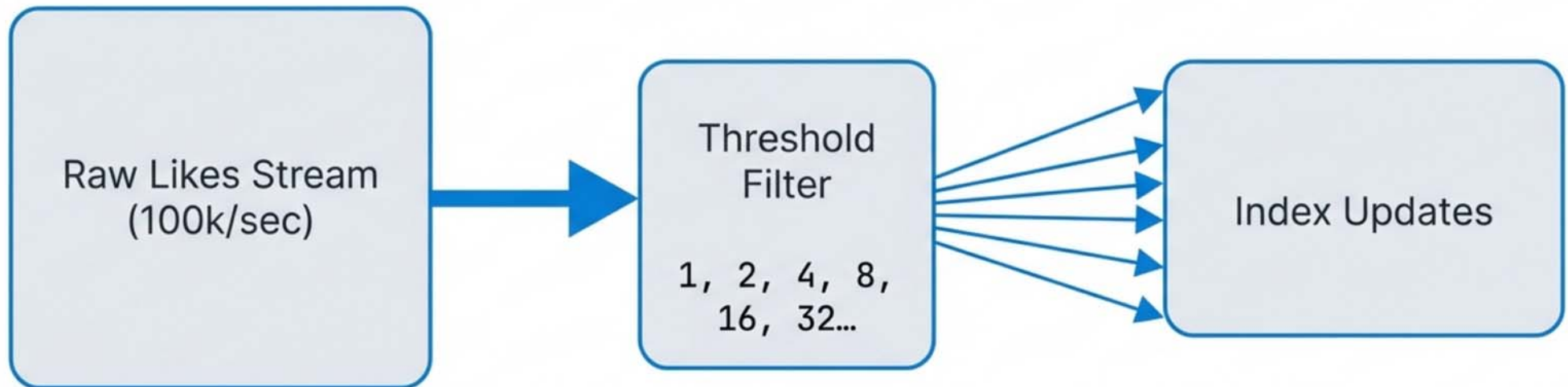


Because search is not personalized, we can aggressively cache results at the edge. A viral event like "Taylor Swift" only hits our backend once per minute per region.

Deep Dive 2: Scaling Writes (The Buffer)



Deep Dive 2 (cont.): Handling the 'Like' Storm



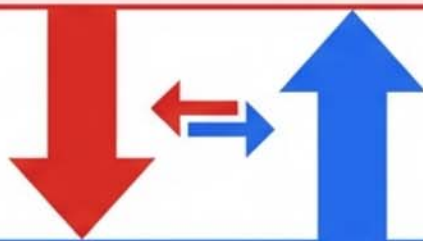
Inter Regular **Approximate** Sorting + Read-Time Correction

1. Index stores approximate score (Power of 2).
2. Search Service fetches top N items.
3. Search Service queries **"Like Cache"** for exact counts and re-sorts.

Deep Dive 3: Storage Optimization (Hot vs Cold)

Hot Storage (Redis)

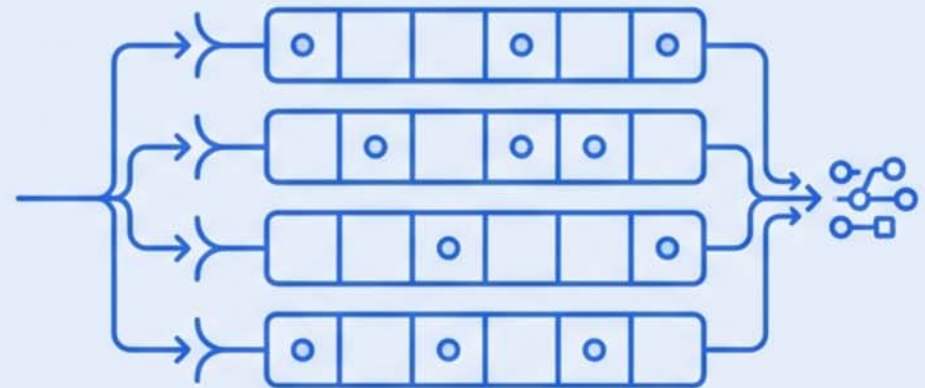
- Top 5% Active Keywords.
- Fast Access, High Cost.



Cold Storage (S3)

- Long-tail, Old Keywords.
- Slower Access, Low Cost.

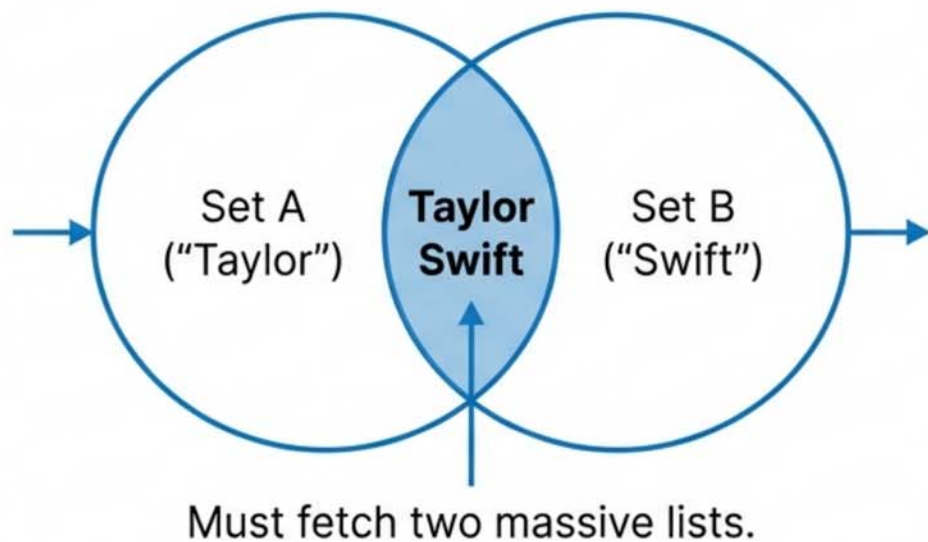
Decision Engine: Count-Min Sketch



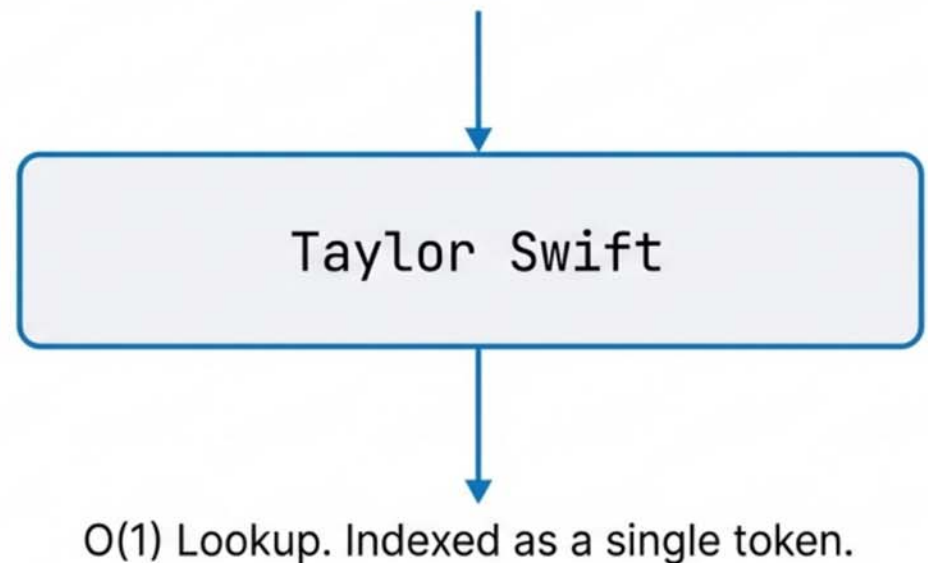
A probabilistic data structure used to track keyword frequency and promote/demote keys between Redis and S3.

Deep Dive 4: Multi-Keyword Search (Bigrams)

Method A: Intersection (Slow)



Method B: Bigrams (Fast)



Storage Trade-off

Use PMI (Pointwise Mutual Information) to only index meaningful pairs JetBrains Mono (e.g., Index 'Taylor Swift', Ignore 'The Video').